

Design and Implementation of Universal Shift Register using Verilog

1 Introduction

A Universal Shift Register is a sequential digital circuit capable of performing multiple operations such as:

- Shift Left
- Shift Right
- Parallel Load
- Hold (No Operation)

Unlike simple shift registers, a universal shift register provides flexibility by supporting both serial and parallel data transfer. It is widely used in digital systems such as microprocessors, data storage systems, communication interfaces, and arithmetic logic units.

The implementation of such a register using Verilog Hardware Description Language (HDL) allows simulation, synthesis, and efficient hardware realization using FPGA or ASIC technologies.

2 Components Needed

The following components are required for designing a Universal Shift Register:

- Flip-Flops (D Flip-Flops are commonly used)
- Multiplexers (for selecting operations)
- Clock signal
- Control inputs (mode selection)
- Serial input lines (left and right)
- Parallel input lines

In Verilog implementation, these components are modeled using behavioral or structural constructs rather than physical components.

3 Inputs and Outputs

3.1 Inputs

- `clk` : Clock signal
- `reset` : Reset signal (synchronous or asynchronous)
- `mode[1:0]` : Control signal to select operation
- `data_in[3:0]` : Parallel input data
- `serial_in_left` : Serial input for left shift
- `serial_in_right` : Serial input for right shift

3.2 Outputs

- `data_out[3:0]` : Output of the register

4 Truth Table

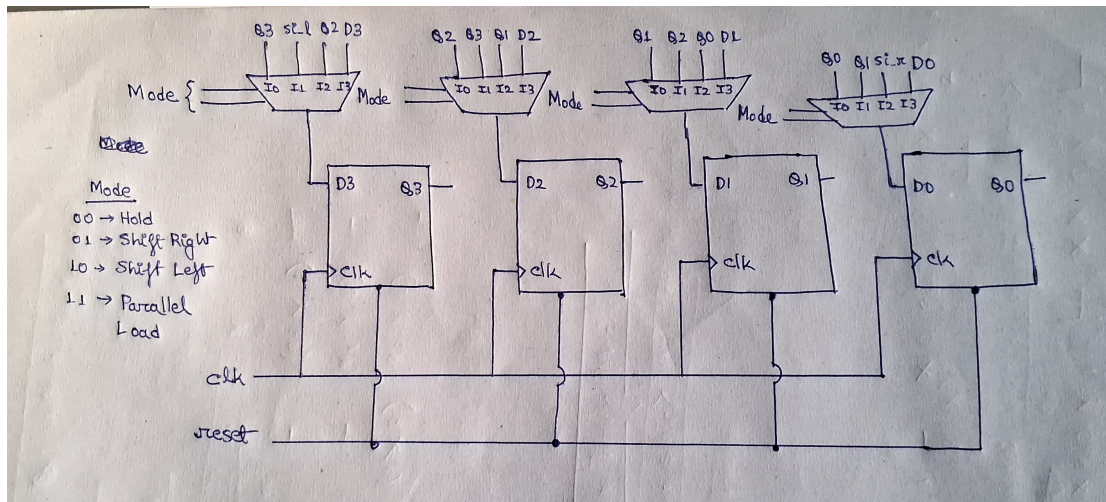
Mode	Operation	Description
00	Hold	No change in output
01	Shift Right	Bits shift right, MSB gets <code>serial_in_left</code>
10	Shift Left	Bits shift left, LSB gets <code>serial_in_right</code>
11	Parallel Load	Load <code>data_in</code> into register

4.1 State Representation

Let the current state be $Q = [Q_3 Q_2 Q_1 Q_0]$

- Shift Right: $Q \leftarrow [serial_in_left, Q_3, Q_2, Q_1]$
- Shift Left: $Q \leftarrow [Q_2, Q_1, Q_0, serial_in_right]$
- Parallel Load: $Q \leftarrow data_in$
- Hold: $Q \leftarrow Q$

4.2 Circuit Diagram



5 Working Principle

The Universal Shift Register operates based on the control signal `mode[1:0]` which determines the functionality at each clock pulse.

- **Hold (00):** The register maintains its previous state. No data movement occurs.
- **Shift Right (01):** Each bit shifts one position to the right. The leftmost bit (MSB) is filled by the serial input `serial_in_left`.
- **Shift Left (10):** Each bit shifts one position to the left. The rightmost bit (LSB) is filled by the serial input `serial_in_right`.
- **Parallel Load (11):** All bits of the register are simultaneously updated with the parallel input `data_in`.

The operation is synchronized with the clock signal. On each rising edge of the clock, the selected operation is executed.

Reset initializes the register to a known state (usually zero).

6 Verilog Design Code

```
module universal_shift_register(  
    input clk,  
    input reset,  
    input [1:0] mode,  
    input [3:0] data_in,  
    input serial_in_left,  
    input serial_in_right,  

```

```

        output reg [3:0] data_out
    );

    always @(posedge clk or posedge reset) begin
        if (reset)
            data_out <= 4'b0000;
        else begin
            case(mode)
                2'b00: data_out <= data_out; // Hold
                2'b01: data_out <= {serial_in_left, data_out[3:1]}; //
                    Shift Right
                2'b10: data_out <= {data_out[2:0], serial_in_right}; //
                    Shift Left
                2'b11: data_out <= data_in; // Parallel Load
            endcase
        end
    end

endmodule

```

7 Testbench Code

```

module tb_universal_shift_register;

    reg clk;
    reg reset;
    reg [1:0] mode;
    reg [3:0] data_in;
    reg serial_in_left;
    reg serial_in_right;
    wire [3:0] data_out;

    universal_shift_register uut (
        .clk(clk),
        .reset(reset),
        .mode(mode),
        .data_in(data_in),
        .serial_in_left(serial_in_left),
        .serial_in_right(serial_in_right),
        .data_out(data_out)
    );

    always #5 clk = ~clk;

    initial begin
        clk = 0;
    end

```

```

    reset = 1;
    #10 reset = 0;

    mode = 2'b11;
    data_in = 4'b1010;
    #10;

    mode = 2'b01;
    serial_in_left = 1;
    #10;

    mode = 2'b10;
    serial_in_right = 0;
    #10;

    mode = 2'b00;
    #10;

    $finish;
end

endmodule

```

8 Code Explanation

8.1 Design Code Explanation

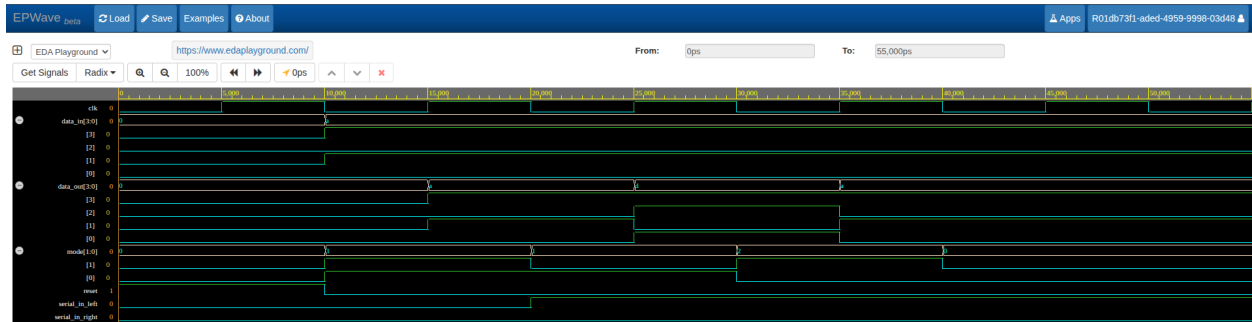
- The module implements a 4-bit universal shift register using behavioral modeling in Verilog.
- The `always` block is sensitive to the rising edge of the clock and the reset signal:
 - On `reset = 1`, the register is initialized to 0000.
 - On each rising edge of `clk`, the operation is selected based on the `mode` input.
- The `mode[1:0]` input determines the operation:
 - 00 : Hold (no change in output)
 - 01 : Shift Right
 - 10 : Shift Left
 - 11 : Parallel Load
- The concatenation operator `{}` is used to implement shifting:
 - Shift Right: `{serial_in_left, data_out[3:1]}`

- Shift Left: `{data_out[2:0], serial_in_right}`
- During shift operations:
 - In Shift Right, the most significant bit (MSB) is filled with `serial_in_left`, and all bits move one position to the right.
 - In Shift Left, the least significant bit (LSB) is filled with `serial_in_right`, and all bits move one position to the left.
- All operations are synchronous, ensuring that the output changes only at the rising edge of the clock, avoiding glitches and ensuring stable sequential behavior.

8.2 Testbench Explanation

- The testbench initializes all input signals to avoid undefined (X) states, ensuring accurate simulation results.
- A clock signal is generated using an `always` block that toggles every 5 time units, resulting in a clock period of 10 time units.
- Waveform dumping is enabled using:
 - `$dumpfile("dump.vcd")`
 - `$dumpvars(0, tb_universal_shift_register)`
- Inputs are applied synchronously using `@(negedge clk)` to ensure they are stable before the active rising edge of the clock, satisfying setup time requirements.
- The sequence of operations performed is:
 - Reset the register to 0000
 - Parallel Load of 1010
 - Shift Right with serial input 1
 - Shift Left with serial input 0
 - Hold operation
- Each operation is executed on the subsequent rising edge of the clock, and the output is observed accordingly.
- The simulation is terminated using `$finish`.

9 Timing Diagram



10 Timing Diagram Explanation

The timing diagram represents the sequential operation of the universal shift register with respect to the clock signal.

- All changes in the output `data_out` occur strictly at the rising edge of the clock, confirming correct synchronous operation.
- Initially, when `reset` = 1, the output is forced to 0000.
- During the Parallel Load operation (`mode` = 11), the input 1010 is loaded into the register in a single clock cycle.
- During Shift Right (`mode` = 01):
 - Bits move one position to the right.
 - The MSB is filled with `serial_in_left`.
 - Example: 1010 $\rightarrow$ 1101
- During Shift Left (`mode` = 10):
 - Bits move one position to the left.
 - The LSB is filled with `serial_in_right`.
 - Example: 1101 $\rightarrow$ 1010
- During Hold operation (`mode` = 00), the output remains unchanged across clock cycles.
- The absence of undefined (X) states and the clean transitions indicate proper initialization and correct testbench design.

11 Conclusion

The Universal Shift Register is a versatile sequential circuit that supports multiple data operations within a single design. Using Verilog, the implementation becomes efficient, scalable, and suitable for simulation and synthesis. This design demonstrates the importance of control logic and sequential behavior in digital systems.